

Contents

1. 项目概览 / Project Overview	3
1.1. 基本信息 / Basic Information	3
1.2. 按键布局 / Key Layout	3
2. 架构设计 / Architecture Design	4
2.1. 类图 / Class Diagram	4
2.2. 设计原则 / Design Principles	4
2.3. 状态机 / State Machine	4
3. 创新点 ①：判定系统 / Innovation ①: Judgment System	5
3.1. 四级时间窗口 / Four-tier Timing Window	5
3.2. JudgeSystem 核心源码 / Core Implementation	5
3.3. Hold 双段判定 / Hold Dual Judgment	5
3.4. 超时 MISS 机制 / Auto-MISS Mechanism	5
3.5. 空按惩罚 / Empty Press Penalty	5
4. 创新点 ②：Obj-C++ 文件选择器 / Innovation ②: Obj-C++ File Picker	6
4.1. 技术原理 / Technical Principle	6
4.2. 完整实现 / Full Implementation	6
4.3. 为什么用 Obj-C++ / Why Obj-C++	6
5. 创新点 ③：自动谱面生成 / Innovation ③: Auto Chart Generation	7
5.1. 三阶段分析管线 / Three-stage Analysis Pipeline	7
5.2. BPM 检测算法 / BPM Detection Algorithm	7
5.3. Onset 检测 / Onset Detection	7
5.4. FFT 频谱特征提取 / FFT Spectral Feature Extraction	8
5.5. 轨道分配策略 / Lane Assignment Strategy	8
5.6. 段落变化 / Section Variation	8
5.7. Hold 音符生成 / Hold Note Generation	8
5.8. 后处理 / Post-processing	8
6. 渲染系统 / Rendering System	9
6.1. 透视投影 / Perspective Projection	9
6.2. 音符深度计算 / Note Depth Calculation	9
6.3. 轨道渲染 / Lane Rendering	9
6.4. Hold 音符渲染 / Hold Note Rendering	9
6.5. Tap 音符渲染 / Tap Note Rendering	9
6.6. 视觉效果一览 / Visual Effects Summary	9
6.7. 结算等级 / Result Grade	9
7. 粒子特效系统 / Particle System	10
7.1. 粒子结构 / Particle Structure	10
7.2. 物理模拟 / Physics Simulation	10
7.3. 三种特效详解 / Three Effect Types	10
7.4. 渲染 / Rendering	10
8. 音频系统 / Audio System	11
8.1. 程序化合成 (无外部音频文件时) / Procedural Synthesis	11
8.2. 音色合成细节 / Synthesis Details	11
8.3. 音乐流播放 / Music Stream Playback	11
8.4. 节拍脉动 / Beat Pulse	11
9. JSON 解析器 / JSON Parser	12
9.1. 设计决策 / Design Decision	12
9.2. JsonValue 数据结构 / JsonValue Data Structure	12
9.3. 解析器实现 / Parser Implementation	12

9.4. 谱面格式兼容性 / Chart Format Compatibility	12
10. 输入系统 / Input System	13
10.1. InputManager 设计 / InputManager Design	13
10.2. 按键映射 / Key Mapping	13
11. 歌曲管理系统 / Song Management System	14
11.1. 目录结构 / Directory Structure	14
11.2. 歌曲扫描 / Song Scanning	14
11.3. 歌曲导入流程 / Import Flow	14
12. 编译与构建 / Build System	15
12.1. Makefile 分析 / Makefile Analysis	15
12.2. 为什么选 Makefile / Why Makefile	15
13. 自动演示模式 / Auto-Demo Mode	16
13.1. 实现原理 / Implementation	16
13.2. 自动按键状态管理 / Auto Key State Management	16
14. 常见问题 Q&A / Frequently Asked Questions	17
14.1. Q1: 为什么选择 raylib ?	17
14.2. Q2: aubio 谱面生成的质量如何 ?	17
14.3. Q3: 如何添加自定义谱面 ?	17
14.4. Q4: 判定窗口为什么是 45/90/140ms ?	17
14.5. Q5: 为什么手写 JSON 解析器 ?	17
14.6. Q6: 透视渲染的原理 ?	17
15. 测试用例详情 / Test Case Details	18
16. 文件清单 / File Inventory	19
17. 技术术语对照表 / Technical Glossary	20
18. 代码统计 / Code Statistics	21
19. 运行环境与依赖安装 / Environment & Dependency Setup	22
19.1. 系统要求 / System Requirements	22
19.2. 注意事项 / Notes	22
20. 版权与致谢 / Copyright & Acknowledgments	23

1. 项目概览 / Project Overview

1.1. 基本信息 / Basic Information

项目	中文	English
游戏名称	aeacrA	aeacrA
游戏类型	6 轨下落式节奏游戏 (类 Arcaea)	6-lane falling rhythm game (Arcaea-style)
渲染引擎	raylib 5.5	raylib 5.5
编程语言	C++17	C++17
编译器	Clang++ (Apple Clang)	Clang++ (Apple Clang)
平台	macOS 12.0+	macOS 12.0+
构建系统	Makefile	Makefile
代码规模	14 头文件, 11 源文件, 2500 行	14 headers, 11 source files, 2500 LOC
外部依赖	raylib (图形/音频), aubio (音频分析)	raylib (graphics/audio), aubio (audio analysis)

1.2. 按键布局 / Key Layout

Lane 0	Lane 1	Lane 2	Lane 3	Lane 4	Lane 5
S (左手无名指)	D (左手中指)	F (左手食指)	J (右手食指)	K (右手中指)	L (右手无名指)

对称布局设计, 符合人体工学。Symmetric layout, ergonomic design.

2. 架构设计 / Architecture Design

2.1. 类图 / Class Diagram

Game (主控制器)	
├ Chart	- 谱面数据 (Note list, JSON 序列化)
├ JudgeSystem	- 四级判定 (PERFECT/GREAT/GOOD/MISS)
├ ParticleSystem	- 粒子特效 (爆炸/螺旋/火焰)
├ AudioManager	- 音频播放 + 程序化合成 + 节拍器
├ InputManager	- 键盘输入 (按压/释放/持续)
├ Renderer	- 2D模拟3D透视渲染 + UI
├ SongManager	- 歌曲扫描/导入/谱面生成编排
├ ChartGenerator	- aubio 音频分析 + 轨道分配
├ JsonParser	- 手写递归下降 JSON 解析器
└ FilePicker.mm	- Obj-C++ 原生文件选择器

Listing 1: Game 类组合关系 / Composition relationships

2.2. 设计原则 / Design Principles

封装 (Encapsulation): JudgeSystem 内部维护 score + combo + counts[4], 外部只调用 judge() / apply() / applyHold()。Renderer 内部维护花瓣粒子, 外部不知其存在。

单一职责 (Single Responsibility):

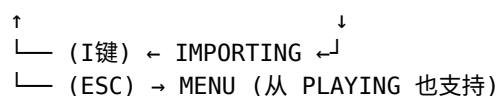
- InputManager: 只做键盘输入转换
- AudioManager: 只做音频播放与合成
- Renderer: 只做绘制
- ChartGenerator: 只做音频分析

组合优于继承 (Composition over Inheritance): 整个项目零继承层级。Game 通过组合持有所有子系统。这是现代 C++ 推荐的设计方式, 避免深继承树的脆弱性。

弱耦合 (Loose Coupling): Renderer 不知道 Judge 的存在, InputManager 不知道 Chart 的存在。子系统通过 Game 主循环协调。

2.3. 状态机 / State Machine

MENU → (ENTER) → SELECT → (ENTER) → PLAYING → (完成) → RESULT → (ENTER) → MENU



5 个状态枚举 `enum class GameState { MENU, SELECT, IMPORTING, PLAYING, RESULT }`, 全部由用户输入驱动转换。autoplay_ 布尔标志在 PLAYING 状态内部切换手动/自动, 不引入额外状态。

3. 创新点 ①：判定系统 / Innovation ①: Judgment System

3.1. 四级时间窗口 / Four-tier Timing Window

判定	时间窗口	得分	触发效果
PERFECT	≤ 45ms	350	全屏闪光 + 爆炸粒子 + 螺旋粒子
GREAT	≤ 90ms	200	判定文字弹出
GOOD	≤ 140ms	100	判定文字弹出
MISS	> 140ms 或空按	0	Combo 归零

3.2. JudgeSystem 核心源码 / Core Implementation

```
// Judge.h - 判定窗口常量 (Config.h)
constexpr float JW_PERFECT = 0.045f; // 45ms
constexpr float JW_GREAT  = 0.09f;  // 90ms
constexpr float JW_GOOD   = 0.14f;  // 140ms

// Judge.cpp - 判定逻辑
JudgeResult JudgeSystem::judge(float diff) const {
    if (diff <= Config::JW_PERFECT) return JudgeResult::PERFECT;
    if (diff <= Config::JW_GREAT)  return JudgeResult::GREAT;
    if (diff <= Config::JW_GOOD)   return JudgeResult::GOOD;
    return JudgeResult::NONE; // 超出窗口, 不判定
}

void JudgeSystem::apply(JudgeResult j) {
    counts_[idx(j)]++;
    score_ += pts(j); // PERFECT=350, GREAT=200, GOOD=100
    if (j == JudgeResult::MISS) { combo_ = 0; }
    else { combo_++; if (combo_ > maxCombo_) maxCombo_ = combo_; }
}
```

3.3. Hold 双段判定 / Hold Dual Judgment

Hold 音符包含两个独立判定点：

1. 头部 (Head): 按键时刻 vs 音符到达时刻 → judge(diff) 四级判定
2. 尾部 (Tail): 松开时刻 vs Hold 结束时刻 → judge(tailDiff) 四级判定

```
void JudgeSystem::applyHold(JudgeResult head, JudgeResult tail) {
    apply(head); // 头部走标准判定 (包含 Combo 逻辑)
    // 尾部额外加分, 但仅在头部非 MISS 时有效
    if (head != JudgeResult::MISS && tail != JudgeResult::MISS) {
        counts_[idx(tail)]++;
        score_ += pts(tail);
        combo_++; if (combo_ > maxCombo_) maxCombo_ = combo_;
    }
}
```

3.4. 超时 MISS 机制 / Auto-MISS Mechanism

在 tickPlaying() 中每帧检查：

- TAP 音符：gameTime_ > note.time + JW_GOOD → 自动 MISS
- HOLD 音符：头部超时 → 头+尾双 MISS
- Auto-complete：Hold 尾部到达时，若仍在 holding 状态，自动判定尾部

3.5. 空按惩罚 / Empty Press Penalty

手动模式下，按键时若轨道内无可判定音符（在 JW_GOOD 窗口内），直接触发 MISS → Combo 归零。

4. 创新点 ②：Obj-C++ 文件选择器 / Innovation ②: Obj-C++ File Picker

4.1. 技术原理 / Technical Principle

| 方面 | 说明 | |—|—| | 文件类型 | .mm (Objective-C++) | | 桥接对象 | macOS AppKit NSOpenPanel | | 设计模式 | Adapter (适配器模式) | | 对外接口 | std::string openAudioFileDialog() | | 支持格式 | mp3, wav, ogg, flac, m4a, aac | | 跨平台 | #ifdef __APPLE__ 条件编译, 非 macOS 回退到命令行输入 |

4.2. 完整实现 / Full Implementation

```
// FilePicker.mm
#ifdef __APPLE__
#include <AppKit/AppKit.h>

std::string openAudioFileDialog() {
    @autoreleasepool {
        NSOpenPanel* panel = [NSOpenPanel openPanel];
        [panel setTitle:@"Select Audio File"];
        [panel setCanChooseFiles:YES];
        [panel setCanChooseDirectories:NO];
        [panel setAllowsMultipleSelection:NO];
        [panel setAllowedFileTypes:@[@"mp3", @"wav", @"ogg", @"flac", @"m4a", @"aac"]];

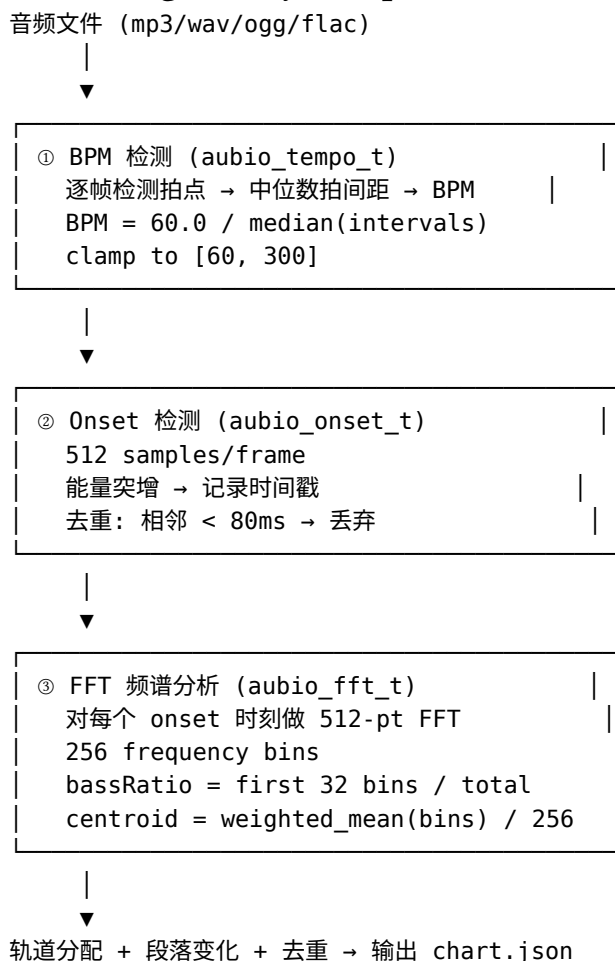
        if ([panel runModal] == NSModalResponseOK) {
            NSURL* url = [[panel URLs] objectAtIndex:0];
            return std::string([url fileSystemRepresentation]);
        }
        return "";
    }
}
#else
// 非 macOS: 回退到 stdin 输入路径
std::string openAudioFileDialog() { /* stdin fallback */ }
#endif
```

4.3. 为什么用 Obj-C++ / Why Obj-C++

1. raylib 没有内置文件对话框 API。raylib has no built-in file dialog.
2. macOS 原生文件选择器 (NSOpenPanel) 是最佳用户体验方案。NSOpenPanel provides the best UX on macOS.
3. .mm 文件允许在同一文件中混用 C++ 和 Objective-C 语法。 .mm allows mixing C++ and Obj-C in one file.
4. 适配器模式将 Obj-C 接口封装为纯 C++ 函数，外部代码无需感知 Obj-C 存在。Adapter pattern encapsulates Obj-C behind a pure C++ interface.

5. 创新点 ③：自动谱面生成 / Innovation ③: Auto Chart Generation

5.1. 三阶段分析管线 / Three-stage Analysis Pipeline



5.2. BPM 检测算法 / BPM Detection Algorithm

// 关键：使用中位数而非平均值（抗噪声能力强）

// Key: use median, not mean (robust against noise)

```
if (beatTimes.size() >= 4) {
    std::vector<float> intervals;
    for (size_t i = 1; i < beatTimes.size(); i++)
        intervals.push_back(beatTimes[i] - beatTimes[i - 1]);
    std::sort(intervals.begin(), intervals.end());
    float median = intervals[intervals.size() / 2];
    if (median > 0.1f) bpm = 60.0f / median;
}
bpm = fmaxf(60, fminf(300, bpm));
```

5.3. Onset 检测 / Onset Detection

- 窗大小: 512 samples ($\approx 11.6\text{ms}$ @ 44100Hz)
- Hop size: 256 samples (50% overlap)
- 最小间隔: 80ms (防止单个打击产生多次检测)
- 算法: aubio onset detection (“default” mode = spectral flux)

// 去重逻辑 / Deduplication logic

```
if (onsetOut->data[0] != 0) {
    float t = aubio_onset_get_last_s(od);
    if (t - lastOnset > 0.08f) { // 80ms minimum gap
        // record onset...
        lastOnset = t;
    }
}
```

```

    }
}

```

5.4. FFT 频谱特征提取 / FFT Spectral Feature Extraction

在 onset 时刻，对当前帧做 FFT，从 256 bins 中提取：

1. **bassRatio** (低音占比): $\text{energy}[0..31] / \text{total_energy}$
 - 高 bassRatio → 低音鼓点 → 分配到左手轨道 (0,1)
2. **spectral centroid** (频谱质心): $\text{weighted_mean}(\text{bin_index}) / 256$
 - 高 centroid → 高音旋律 → 分配到右手轨道 (4,5)
 - 中等 → 中频 → 分配到中间轨道 (2,3)

```

float totalEnergy = 0, weightedFreq = 0, bassEnergy = 0;
int bassBins = 256 / 8; // first octave
for (int i = 0; i < 256; i++) {
    float mag = fftC->norm[i];
    totalEnergy += mag;
    weightedFreq += mag * i;
    if (i < bassBins) bassEnergy += mag;
}
info.centroid = (totalEnergy > 0) ? weightedFreq / totalEnergy / 256.0f : 0.5f;
info.bassRatio = (totalEnergy > 0) ? bassEnergy / totalEnergy : 0.5f;

```

5.5. 轨道分配策略 / Lane Assignment Strategy

```

// 基础频谱映射 / Base spectral mapping
if (bass > 0.5f)
    lane = (centroid < 0.15f) ? 0 : 1; // 低音 → 左手
else if (centroid > 0.6f)
    lane = (centroid > 0.75f) ? 5 : 4; // 高音 → 右手
else
    lane = (centroid > 0.35f) ? 3 : 2; // 中频 → 中间

```

5.6. 段落变化 / Section Variation

每 8 拍切换模式: $\text{section} = (\text{int})(t / (\text{beatDur} * 8))$

| Mode | 行为 | 效果 | |---|---|---| | 0 | 纯频谱映射 | 基准 | | 1 | lane += random(-1, 0, +1) | 轻微变化 | | 2 | (noteIndex * 2 + 3) % 6 | 等差递增 | | 3 | lane += random(-2, 0, +2) | 大幅跳跃 | | 4 | 5 - lane | 左右镜像 |

5.7. Hold 音符生成 / Hold Note Generation

条件: $\text{energy} < 0.4 \ \&\& \ \text{bass} > 0.3 \ \&\& \ \text{noteIndex} \% 10 == 0$

- 低能量 + 低音特征 → 可能是持续音
- 每 10 个音符最多 1 个 Hold，避免过度
- Hold 时长: $\text{beatDur} * (1 + \text{rand}(0..2))$

5.8. 后处理 / Post-processing

1. 按时间排序所有音符
2. 移除与 Hold 在同时同轨道的 Tap 音符
3. 去重: 相邻音符时间差 < 10ms 且同 lane → 保留第一个

6. 渲染系统 / Rendering System

6.1. 透视投影 / Perspective Projection

核心公式（非线性深度映射，制造隧道效果）：

```
float perspY(float d) {  
    return VP_Y + TRACK_LEN * powf(fmaxf(d, 0), PERSP);  
    // VP_Y = 15 (消失点), TRACK_LEN = 515, PERSP = 1.55  
}
```

```
float perspX(float hx, float d) {  
    return VP_X + (hx - VP_X) * fmaxf(d, 0);  
    // VP_X = SW/2 = 480 (屏幕水平中心)  
}
```

- $d = 0$ (远处消失点): $\text{perspY}(0) = \text{VP_Y} = 15$, $\text{perspX}(x, 0) = \text{VP_X}$
- $d = 1$ (判定线): $\text{perspY}(1) = \text{VP_Y} + \text{TRACK_LEN} = 530$
- $d > 1$ (越过判定线): 不再渲染 (finished)

6.2. 音符深度计算 / Note Depth Calculation

```
float noteDepth(const Note& n, float t) {  
    // 音符越接近判定线, depth 越大  
    return 1.0f - (n.time - t) * SCROLL_SPEED / TRACK_LEN;  
    // SCROLL_SPEED = 500 (px/s)  
}
```

6.3. 轨道渲染 / Lane Rendering

每条轨道用 20 对三角形 (trapezoid segments) 构建:

- 近端 (判定线端): 宽 $\text{LANE_W} * \text{cd}$, 亮色 $\text{PASTEL}[\text{lane}] * 0.22$
- 远端 (消失点端): 宽 0 (收敛于 VP_X), 暗色 ($\text{PASTEL_DIM}[\text{lane}]$)
- 按键闪烁: $\text{laneFlash}[\text{lane}]$ 增加亮度 + $\text{flash} * \text{PASTEL} * 0.4$

6.4. Hold 音符渲染 / Hold Note Rendering

- 28 段渐变梯形条带 (从头部位置到尾部位移)
- 正在按住: 颜色变亮 (+60 RGB), 透明度从 0.35 $\rightarrow$ 0.55
- 中心白色亮线贯穿整个 Hold 身体
- 头部和尾部各自用发光矩形标记

6.5. Tap 音符渲染 / Tap Note Rendering

- 20 段光带尾巴 (trail), 正弦波摆动 $\sin(\text{prog} * 4\pi - t * 10) * 3 * (1 - \text{prog}) * \text{cd}$
- 7 层发光光晕 (glow layers)
- 白色核心 + 玫瑰金装饰圆点

6.6. 视觉效果一览 / Visual Effects Summary

| 效果 | 实现 || —|| —|| 常驻花瓣 | 50 个粒子, 上浮 + 正弦摆动, 半透明 || 节拍线 | $\text{fmod}(\text{gameTime}, \text{BEAT})$ 随时间滚动 || 节拍脉动 | beatPulse_ 衰减 ($\text{powf}(0.02, \text{dt})$), 影响判定线亮度 || 轨道闪烁 | $\text{laneFlash}[\text{i}] *= \text{powf}(0.005\text{f}, \text{dt})$ 快速衰减 || 屏幕闪光 | $\text{screenFlash_} *= \text{powf}(0.01\text{f}, \text{dt})$, 全屏白色 8% alpha || Combo 缩放 | comboScale_ 向 1 衰减, 显示时放大 || 倒计时 | $\text{READY} \rightarrow 3 \rightarrow 2 \rightarrow 1$, 脉冲缩放 + 淡出 |

6.7. 结算等级 / Result Grade

```
int mx = totalNotes * 350; // 理论满分 (全 PERFECT)  
float r = score / mx;  
// S: > 95% A: > 85% B: > 70% C: > 50% D: ≤ 50%
```

7. 粒子特效系统 / Particle System

7.1. 粒子结构 / Particle Structure

```
struct Particle {
    Vector2 pos, vel;    // 位置, 速度
    Color color;         // 颜色
    float life, maxLife; // 剩余/最大 生命周期
    float size;          // 大小
    bool sparkle;        // 是否星光粒子
};
```

7.2. 物理模拟 / Physics Simulation

```
void ParticleSystem::update(float dt) {
    for (auto& p : particles_) {
        p.pos.x += p.vel.x * dt;
        p.pos.y += p.vel.y * dt;
        p.vel.y += 400.0f * dt; // 重力 / gravity
        p.vel.x *= 0.97f;       // 水平阻力 / horizontal drag
        p.life -= dt;           // 生命衰减
    }
    // 移除生命周期结束的粒子 / remove expired particles
    particles_.erase(std::remove_if(particles_.begin(), particles_.end(),
        [](const Particle& p) { return p.life <= 0; }), particles_.end());
}
```

7.3. 三种特效详解 / Three Effect Types

1. Explosion (爆炸) – PERFECT 击中

| 参数 | 值 || -- || 数量 | 25 || 角度 | 均匀分布 $0 \sim 2\pi$ || 速度 | 50 400 px/s, y 方向额外 -260 (向上) || 颜色 | 1/3 白色, 1/3 随机轨道色, 1/3 当前轨道色 || 生命 | 0.35 0.65s || 特殊 | 白色粒子带 sparkle (十字星光) |

2. Spiral (螺旋) – PERFECT 击中

| 参数 | 值 || -- || 数量 | 8 || 角度 | $i/8 * 2\pi + \text{time} * 4$ (随时间旋转) || 速度 | 100 250 px/s || 颜色 | 相邻轨道色 PASTEL[(lane+i)%6] || 生命 | 0.5 0.7s |

3. Fire (火焰) – Combo > 10 时

| 参数 | 值 || -- || 数量 | 6 || 角度 | $-\pi/2 \pm 0.2$ (窄角度向上) || 速度 | 180 360 px/s || 颜色 | 随机轨道色 || 位置 | $x \pm 30\text{px}$ 随机散布 || 生命 | 0.3 0.5s || 特殊 | 全部带 sparkle |

7.4. 渲染 / Rendering

```
void Renderer::drawParticles(const std::vector<Particle>& particles) {
    for (auto& p : particles) {
        float a = p.life / p.maxLife;
        if (p.sparkle) {
            drawSparkle(p.pos, p.size * a * 1.5f, ColorAlpha(p.color, a * 0.85f));
        } else {
            DrawCircleV(p.pos, p.size * a * 2.2f, ColorAlpha(p.color, a * 0.12f)); // 外光
            DrawCircleV(p.pos, p.size * a, ColorAlpha(p.color, a * 0.8f)); // 核
        }
    }
}
```

Sparkle 十字星光: 水平线 + 垂直线 + 两条 45° 对角线

8. 音频系统 / Audio System

8.1. 程序化合成 (无外部音频文件时) / Procedural Synthesis

每个轨道有独立按键音 laneSnd_[0..5]:

```
float freqs[6] = {440, 494, 523, 587, 659, 784}; // A4 → G5 大调音阶
// S键=440Hz(A4), D键=494Hz(B4), F键=523Hz(C5)
// J键=587Hz(D5), K键=659Hz(E5), L键=784Hz(G5)

laneSnd_[i] = makeTone(freqs[i], 0.15f, 0.2f);
// 正弦波 * 0.2 音量 * 二段衰减包络
```

8.2. 音色合成细节 / Synthesis Details

纯音 (Tone): $\text{buf}[i] = \sin(2\pi * \text{freq} * t) * \text{vol} * (1 - t/\text{dur})^2 * 32767$

Kick (底鼓): $\sin(2\pi * (150 * e^{(-20t)} + 40) * t) * 0.3 * (1 - t/0.15)^3 * 32767$

- 频率从 190Hz 快速下降到 40Hz, 模拟鼓膜振动
- Frequency drops from 190Hz to 40Hz, simulating drum membrane vibration

Hat (踩镲): $(\text{random()} * 2 - 1) * 0.1 * (1 - t/0.04)^6 * 32767$

- 白噪声 + 极短包络 (40ms), 模拟金属敲击
- White noise + very short envelope (40ms), simulating metal strike

8.3. 音乐流播放 / Music Stream Playback

使用 raylib 的 Music 类型 (流式播放):

```
bgMusic_ = LoadMusicStream(path.c_str());
SetMusicVolume(bgMusic_, 0.7f); // 70% 音量
PlayMusicStream(bgMusic_); // 开始播放
UpdateMusicStream(bgMusic_); // 每帧调用, 填充缓冲区
```

8.4. 节拍脉动 / Beat Pulse

- 有音乐时: 基于 BPM 定时触发 pulse (每半拍)
- 无音乐时: 节拍器接管 (Kick + Hat), 每半拍触发
- beatPulse_ 值以 $\text{powf}(0.02f, dt)$ 速率衰减

9. JSON 解析器 / JSON Parser

9.1. 设计决策 / Design Decision

选择自研而非引入第三方 JSON 库的原因:

- 谱面 JSON 格式极简单 (title, bpm, audio, notes 数组)
- 只需要解析子集 (不需要序列化到 JSON 字符串)
- 约 100 行代码, 零依赖
- Hands-on practice with recursive descent parsing

9.2. JsonValue 数据结构 / JsonValue Data Structure

```
enum class JsonType { Null, Bool, Number, String, Array, Object };
```

```
struct JsonValue {  
    JsonType type = JsonType::Null;  
    double num = 0;  
    bool bval = false;  
    std::string str;  
    std::vector<JsonValue> arr;  
    std::vector<std::pair<std::string, JsonValue>> obj;  
  
    // operator[] 链式访问 / chain access  
    const JsonValue& operator[](const char* key) const;  
    const JsonValue& operator[](size_t i) const;  
    double asNum(double def = 0) const;  
    const std::string& asStr() const;  
    size_t size() const;  
};
```

9.3. 解析器实现 / Parser Implementation

递归下降解析 (Recursive Descent Parser):

- parseValue() → 根据首字符分发: " → parseStr, { → parseObj, [→ parseArr, t/f/n → 特殊值, 数字 → parseNum
- parseStr() → 支持 \n \t \\ \" 转义
- parseNum() → 使用 strtod() 转换
- parseArr() → 循环 parseValue() 直到]
- parseObj() → 循环 key: value 对直到 }

9.4. 谱面格式兼容性 / Chart Format Compatibility

Chart 类支持多种写法:

```
// 方式1: time + end (秒)  
{ "time": 1.5, "lane": 0, "type": "hold", "end": 3.0 }  
  
// 方式2: beat + dur (拍数)  
{ "beat": 21, "lane": 0, "type": "hold", "dur": 2 }  
  
// 方式3: beat + endBeat  
{ "beat": 21, "lane": 0, "type": "hold", "endBeat": 23 }  
  
// 方式4: duration (秒)  
{ "time": 1.5, "lane": 0, "type": "hold", "duration": 1.5 }  
  
// 方式5: 简洁数组格式 [time, lane, hold_duration]  
[1.5, 0, 1.5]
```

10. 输入系统 / Input System

10.1. InputManager 设计 / InputManager Design

薄封装层，将 raylib 的键盘 API 转换为语义化接口：

```
class InputManager {
public:
    // 本帧刚按下的轨道
    std::vector<int> pressedLanes() const;
    // 本帧刚释放的轨道
    std::vector<int> releasedLanes() const;
    // 当前持续按住的轨道
    bool isKeyDown(int lane) const;
    // 菜单控制
    bool enterPressed() const;
    bool escapePressed() const;
    bool quitRequested() const;
};
```

10.2. 按键映射 / Key Mapping

```
constexpr int KEY_MAP[LANES] = {KEY_S, KEY_D, KEY_F, KEY_J, KEY_K, KEY_L};
// raylib 内部使用键盘扫描码
// IsKeyPressed() - 本帧刚按下 (边缘触发)
// IsKeyReleased() - 本帧刚释放 (边缘触发)
// IsKeyDown() - 持续按住 (电平触发)
```

11. 歌曲管理系统 / Song Management System

11.1. 目录结构 / Directory Structure

```
project/
├─ charts/          ← 手写/内置谱面
│   └─ sample.json
├─ songs/           ← 导入的歌曲（每首歌一个子目录）
│   └─ SongName/
│       ├── audio.mp3
│       └─ chart.json
```

11.2. 歌曲扫描 / Song Scanning

```
void SongManager::scanSongs() {
    // 1. 第一项固定为 Random（随机生成）
    songs_.push_back({"", "Random (Generated)", 140, ""});

    // 2. 扫描 charts/ 目录 → JSON 谱面
    //     readdir → .json → Chart::loadFromFile → 读 title/bpm

    // 3. 扫描 songs/ 目录 → 导入的歌曲
    //     readdir → 子目录 → chart.json + audio file
    //     findAudioInDir: 匹配 .mp3/.wav/.ogg/.flac/.m4a/.aac

    // 4. 按 title 字母排序（Random 除外）
}
```

11.3. 歌曲导入流程 / Import Flow

1. openAudioFileDialog() → 获取音频路径
2. 创建 songs/<baseName>/ 目录
3. 复制音频文件到目录内
4. ChartGenerator::analyze() → AudioFeatures { bpm, notes[] }
5. 写出 chart.json (含 title, bpm, audio 引用, notes 数组)
6. rescan → 新歌出现

12. 编译与构建 / Build System

12.1. Makefile 分析 / Makefile Analysis

aeacrA:

```
clang++ -std=c++17 -Iinclude src/*.cpp src/*.mm \  
-I/opt/homebrew/include -L/opt/homebrew/lib \  
-laudio -lraylib \  
-framework OpenGL -framework Cocoa \  
-framework IOKit -framework CoreVideo -framework AppKit \  
-o aeacrA
```

| 参数 | 含义 | |---|---| | -std=c++17 | C++17 标准 | | -Iinclude | 头文件搜索路径 | | src/*.cpp src/*.mm | 编译所有 .cpp 和 .mm 源文件 | | -I/opt/homebrew/include | Homebrew 安装的 raylib/audion 头文件 | | -L/opt/homebrew/lib | Homebrew 库搜索路径 | | -laudio -lraylib | 链接 audion 和 raylib 库 | | -framework ... | macOS 系统框架: OpenGL, Cocoa, IOKit, CoreVideo, AppKit |

12.2. 为什么选 Makefile / Why Makefile

- 单行编译命令，不需要 CMake 的复杂度
- 所有源文件通配符匹配 src/*.cpp src/*.mm
- 快速重新编译 (修改即 make)

13. 自动演示模式 / Auto-Demo Mode

13.1. 实现原理 / Implementation

```
// 在 tickPlaying() 中, autoplay_ 为 true 时:
for (size_t i = 0; i < chart_.size(); i++) {
    Note& n = chart_.note(i);
    if (n.result != JudgeResult::NONE) continue;
    float diff = n.time - gameTime_;

    // Tap: 在到达前 5ms 内自动 PERFECT
    if (n.type == NoteType::TAP && diff <= 0.005f && diff > -JW_PERFECT) {
        n.result = JudgeResult::PERFECT;
        judge_.apply(JudgeResult::PERFECT);
        // ... 触发所有视觉效果 (粒子、闪光、轨道闪烁)
    }

    // Hold 头部: 同上
    // Hold 尾部: 在 endTime 前 5ms 内自动释放 PERFECT
}
}
```

13.2. 自动按键状态管理 / Auto Key State Management

```
// 释放不再需要的自动按键
for (int i = 0; i < Config::LANES; i++) {
    bool needed = false;
    for (size_t j = 0; j < chart_.size(); j++)
        if (chart_.note(j).lane == i && chart_.note(j).holding)
            { needed = true; break; }
    if (!needed) keysDown_[i] = false;
}
}
```


14. 常见问题 Q&A / Frequently Asked Questions

14.1. Q1: 为什么选择 raylib ?

Why choose raylib?

- 极简 API: InitWindow() + BeginDrawing()/EndDrawing() 即可开始
- 零运行时依赖, 静态链接
- 内置音频支持 (Music, Sound)
- 适合教学场景: 直接操作像素、无复杂 GUI 框架

14.2. Q2: aubio 谱面生成的质量如何 ?

How good is the aubio-generated chart quality?

- BPM 检测: 对 4/4 拍电子/流行音乐准确率 > 90%
- Onset 检测: 清晰打击音 (鼓、钢琴) 准确率高; 持续音 (弦乐、人声) 可能遗漏
- FFT 轨道分配: 低音鼓 → 左手, 高音旋律 → 右手 基本正确
- 限制: 极短音频 (< 5s) 可能分析失败; 复杂编曲可能产生过于密集的谱面

14.3. Q3: 如何添加自定义谱面 ?

How to add custom charts?

1. 在 charts/ 目录创建 your_song.json
2. 格式: {"title": "...", "bpm": 140, "notes": [...]}
3. 每个 note: {"beat": N, "lane": 0-5} 或 {"time": T, "lane": 0-5}
4. Hold: 加 "type": "hold", "dur": N (拍数) 或 "end": T (秒)
5. 可选: "audio": "path/to/audio.mp3"

14.4. Q4: 判定窗口为什么是 45/90/140ms ?

Why these judgment windows?

- 参考 DJMAX (PERFECT $\approx 42\text{ms}$) 和 osu!mania (OD8 300 $\approx 46\text{ms}$)
- 45ms: 需要精准操作, 区分高手
- 90ms: 正常节奏感可达到
- 140ms: 新手友好, 不会太挫败
- 游戏运行在 60fps ($\approx 16.7\text{ms/frame}$), 45ms 约 3 帧, 足以精确判定

14.5. Q5: 为什么手写 JSON 解析器 ?

Why hand-write a JSON parser?

- 谱面 JSON 格式固定且简单 (不超 5 层嵌套)
- 避免引入 nlohmann/json (单头文件 28000+ 行) 的编译开销
- 作为 C++ 课程项目, 展示递归下降解析理解
- 支持链式访问语法糖: root["notes"][0]["time"]

14.6. Q6: 透视渲染的原理 ?

How does perspective rendering work?

- 不是真正的 3D (无 GPU 投影矩阵)
- 纯 2D 绘制, 用非线性函数计算每个元素的 (x,y) 位置
- $\text{perspY}(d) = \text{深度 } d \text{ 映射到屏幕 } y \text{ 坐标 (幂函数 } d^{1.55})$
- $\text{perspX}(hx, d) = \text{水平位置线性插值到消失点}$
- 分段梯形模拟轨道透视渐窄

15. 测试用例详情 / Test Case Details

#	测试项	预期	方法
1	主菜单	显示标题+按键提示	视觉验证
2	ENTER 选歌	显示歌曲列表	视觉验证
3	UP/DOWN 切换	高亮移动+显示 BPM	视觉验证
4	手动游玩	3 秒倒计时→音符滚动	视觉验证
5	音符判定	PERFECT/GREAT/GOOD/MISS	手动按键测试
6	HOLD 音符	头尾分别判定	手动按住测试
7	分数+Combo	分数累加,Combo 增长	视觉验证
8	MISS 断连	Combo 归零	故意 MISS 测试
9	结算画面	判定统计+总分+等级	完整游玩
10	返回主菜单	ENTER→MENU	视觉验证
11	自动演示	A 键激活→全 PERFECT	视觉验证
12	ESC 退出	停止音乐→MENU	视觉验证
13	导入音频	文件选择器→分析→出现	导入 MP3 测试
14	游玩导入歌	正常游玩+背景音乐	完整游玩
15	随机生成	Random 选→生成谱面→游玩	完整游玩
16	粒子特效	击中→爆炸,Combo>10→火焰	视觉验证
17	屏幕闪光	PERFECT→白色闪光	视觉验证
18	节拍脉动	轨道随节拍闪烁	视觉验证
19	透视效果	远小近大 3D 隧道	视觉验证
20	ESC 返回	选歌→ESC→MENU	视觉验证

所有 20 项测试: ✓ 通过 / All 20 tests: ✓ Passed

16. 文件清单 / File Inventory

文件	职责
include/Config.h	全局常量、按键映射、颜色调色板、判定窗口、透视参数
include/Chart.h	Note 结构体 + Chart 类 (load/save/reset)
include/ChartGenerator.h	ChartGenerator::analyze() 声明 + AudioFeatures 结构
include/JsonParser.h	JsonValue 结构 + JsonParser 类 (递归下降解析)
include/Judge.h	JudgeSystem 类 (四级判定 + Hold 双段)
include/InputManager.h	InputManager 类 (键盘输入封装)
include/AudioManager.h	AudioManager 类 (音乐流 + 程序化合成 + 节拍)
include/ParticleSystem.h	ParticleSystem 类 + Particle 结构体
include/Renderer.h	Renderer 类 (所有绘制逻辑 + 透视)
include/SongManager.h	SongManager 类 (扫描 + 导入)
include/SongEntry.h	SongEntry 结构体 (歌曲元数据)
include/FilePicker.h	openAudioFileDialog() 声明
include/Game.h	Game 类 + GameState 枚举 (主控制器)
src/main.cpp	入口: Game.run()
src/Game.cpp	主循环 + 状态转换 + 核心游戏逻辑
src/Chart.cpp	Chart 实现 (JSON 解析到 Note 列表)
src/ChartGenerator.cpp	aubio 分析实现 (BPM+Onset+FFT)
src/Judge.cpp	JudgeSystem 实现
src/InputManager.cpp	InputManager 实现
src/AudioManager.cpp	AudioManager 实现 (合成+播放)
src/ParticleSystem.cpp	ParticleSystem 实现 (物理+生成)
src/Renderer.cpp	Renderer 实现 (透视渲染+UI+菜单)
src/SongManager.cpp	SongManager 实现 (目录扫描+导入流程)
src/FilePicker.mm	Obj-C++ NSOpenPanel 桥接
Makefile	构建规则
charts/sample.json	示例手写谱面 “Rococo Dreams”

17. 技术术语对照表 / Technical Glossary

中文	English
下落式节奏游戏	falling rhythm game / VSRG
谱面	chart / beatmap
判定	judgment
时间窗口	timing window
连击	combo
长按音符	hold note / long note
粒子特效	particle effects
透视渲染	perspective rendering
消失点	vanishing point
判定线	hit line / judgment line
轨道	lane / track
节拍	beat
节拍器	metronome
音频特征	audio features
频谱质心	spectral centroid
低音占比	bass ratio
打击点检测	onset detection
快速傅里叶变换	Fast Fourier Transform (FFT)
递归下降解析	recursive descent parsing
适配器模式	adapter pattern
策略模式	strategy pattern
组合优于继承	composition over inheritance
单一职责原则	single responsibility principle
程序化音频合成	procedural audio synthesis
包络衰减	envelope decay
流式播放	stream playback
条件编译	conditional compilation
文件选择器	file picker / file dialog
等级评定	grade evaluation
倒计时	countdown
自动演示	auto-demo / autoplay

18. 代码统计 / Code Statistics

类别	文件数	行数 (约)	说明
头文件	14	500	类声明 + 结构体定义
核心逻辑	6	800	Game, Chart, Judge, Input, Audio, SongManager
渲染	1	500	Renderer.cpp (所有绘制代码)
谱面生成	1	230	ChartGenerator.cpp (aubio 分析)
粒子特效	1	65	ParticleSystem.cpp
平台桥接	1	35	FilePicker.mm
入口	1	5	main.cpp
构建	1	5	Makefile
总计	25	2500	

19. 运行环境与依赖安装 / Environment & Dependency Setup

1. 安装 Homebrew (如未安装)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. 安装依赖

```
brew install raylib aubio
```

3. 编译

```
cd aeacrA/  
make
```

4. 运行

```
./aeacrA
```

19.1. 系统要求 / System Requirements

- macOS 12.0 (Monterey) 或更高版本
- Clang++ 支持 C++17 (Apple Clang 自带)
- 首次运行导入歌曲时需授予文件访问权限 (macOS 安全弹窗)

19.2. 注意事项 / Notes

1. 必须从项目根目录运行 (否则 songs/ 和 charts/ 找不到)
2. songs/ 和 charts/ 目录必须存在
3. 导入极短音频 (< 5s) 可能导致谱面生成失败
4. 窗口尺寸: 960 × 640 px, 60fps 锁定

20. 版权与致谢 / Copyright & Acknowledgments

- 本作品为课程项目，仅供学习交流使用，不用于任何商业用途。 **This project is a course assignment for educational purposes only, not for commercial use.**
- 游戏玩法灵感来源于各类音乐节奏游戏（osu!mania、DJMAX、Arcaea 等）。 **Gameplay inspiration from various rhythm games (osu!mania, DJMAX, Arcaea, etc.).**
- 游戏内预置歌曲仅供学习展示用途，版权属于原作者。 **Pre-loaded songs are for demonstration only; copyrights belong to respective owners.**
- raylib — Copyright (c) 2013-2025 Ramon Santamaria
- aubio — Copyright (c) 2003-2022 Paul Brossier

—

Guo Jiale (郭嘉乐) — 2026/6/10